

Combinatorial Coding, Key-Value Recall, and Emergent Constraint Satisfaction in a Hippocampal-Inspired Spiking Network

Gourav

September 9, 2026

Abstract

We study a hippocampal-inspired memory architecture in which solved Sudoku grids are stored as sparse binary engrams, formed by projecting each grid’s 81×81 same-digit coincidence matrix through a fixed random sparse matrix and taking the top- K active units. An attention-based key-value readout retrieves stored digit values with no observed capacity ceiling up to $M = 2000$ stored items and tolerance to 30% query corruption, in sharp contrast to a ridge-regression readout, which collapses to chance performance at $M \sim H$ even without any query noise. Retrieval degrades gracefully, not catastrophically, under heavier query corruption too: accuracy holds to 35% of engram bits flipped and then declines over a narrow window whose rescue by conflict-mask cleanup peaks sharply at 42% noise. Partial-grid cues drive an iterative attractor that recovers missing digits with graceful, clue-count-dependent accuracy. We then ask whether the symbolic legality check that this architecture relies on for error correction – Sudoku’s row/column/box constraint – can instead emerge from neural dynamics rather than being hard-coded. We build a leaky-integrate-and-fire spiking network whose only structural knowledge of Sudoku is which cell-pairs are *forbidden* from sharing a digit (realized as inhibitory coupling) and which are not (realized as weak excitatory coupling). Driven by per-cell currents encoding candidate digit identities, the network’s steady-state spike timing reproduces the coincidence matrix without any digit ever being decoded from a single neuron’s activity: same-digit cells synchronize to within 0.2 ms while differently-digit cells separate by ~ 13 ms, consistently across eight independently sampled grids. Reading out this emergent, synchrony-based coincidence matrix and feeding it through the *same* associative engram pipeline used for storage yields completion accuracy that is statistically indistinguishable (Wilcoxon signed-rank, $p > 0.05$ at every tested clue fraction, paired $n = 20$) from the original hard-coded conflict mask, while substantially exceeding the accuracy of the raw single-shot associative guess it starts from. This dynamical mechanism is itself robust to noise: corrupting known cells’ drive currents with Gaussian jitter degrades completion accuracy smoothly, remaining above the noise-free associative baseline even at current noise more than twice the spacing between adjacent digit levels. We also characterize a reproducible power-law crossover in attractor relaxation time near the completion threshold, and report – rather than paper over – that its exponent is not universal across stored-set sizes, tempering any claim to a shared critical-phenomena universality class. Finally, we extend the engram layer to a second modality – natural images, heteroassociated to Sudoku engrams via the same bidirectional pseudo-inverse rule used by Vector-HaSH [11] – and benchmark it against classical Hopfield networks. The comparison is double-edged: a paired nearest-neighbor baseline beats our pipeline at every tested stored-set size, and the gap widens rather than closes as that size grows (+3.5 to +96.0 percentage points from $M=200$ to $M=2000$), an honest limitation we report rather than omit; but on the specific property Vector-HaSH is designed to provide, our architecture succeeds where classical and even pseudo-inverse-trained Hopfield networks – the latter using the identical learning rule we do, and statistically tied with ours in noise tolerance at fixed storage size – collapse to zero recovery an order of magnitude sooner in storage capacity, isolating the separate error-correcting attractor, not the learning

rule, as the source of the graceful degradation. All code, data, and reproduction instructions accompany this paper.

1 Introduction

Biological memory systems, and the hippocampus in particular, are widely understood to store information as sparse, combinatorial codes and to retrieve it through content-addressable, key-value-like association rather than by address [1, 2, 3]. A recurring computational question for such architectures is how they resolve *ambiguity*: when a retrieval cue is noisy, partial, or internally inconsistent, some mechanism must clean it up before or during recall. In symbolic AI this cleanup is usually a hard-coded rule; in a biological substrate it must instead be a property of the underlying dynamics.

We use Sudoku as a controlled testbed for this question. A solved Sudoku grid is a natural combinatorial object: 81 cells, each carrying one of nine digits, subject to the constraint that no two cells sharing a row, column, or 3×3 box carry the same digit. This gives a well-defined, tunable notion of query difficulty (the fraction of cells given as clues) and an unambiguous ground truth against which any completion can be checked.

We build our architecture in three layers, corresponding to three literatures we intend this work to speak to:

- **Combinatorial coding.** Each stored grid’s relational structure (which cell-pairs share a digit) is projected through a fixed random sparse matrix and sparsified to a k -winner-take-all binary engram – a scheme in the spirit of Willshaw associative memory [4] and sparse random-projection coding.
- **Key-value memory.** Retrieval uses an attention-style readout – softmax similarity between a query engram and all stored engrams, weighting the corresponding stored values – rather than a single learned linear map, echoing the connection between classical and modern dense associative memory and Transformer-style attention [5, 6].
- **Constraint satisfaction as dynamics.** We ask whether Sudoku’s legality constraint, which the architecture above uses explicitly as a hard-coded cleanup rule, can instead be realized as an emergent property of a spiking network’s synchronization dynamics – a question in the spirit of the broader statistical mechanics of constraint satisfaction and random k -SAT [7, 8], which we invoke here as motivating context rather than as a universality result we claim to extend.

We additionally ask, in Section 4, whether the engram layer generalizes beyond a single content type: we heteroassociate natural images to Sudoku engrams using the bidirectional pseudo-inverse mechanism of Vector-HaSH [11], and benchmark the result directly against classical Hopfield networks on identical data.

Our central empirical result is that these three layers compose: a physically-implemented, dynamics-based legality check, built entirely from local excitatory/inhibitory coupling with no reference to Sudoku’s symbolic rules beyond *which cells must differ*, is statistically indistinguishable from the hard-coded rule it replaces, once routed through the same combinatorial-coding and key-value-recall machinery. We view the value of this result as existence-proof and mechanism, not as an improvement over a rule that was never computationally difficult to hard-code in the first place.

2 Model

2.1 Coincidence-matrix engram and key-value readout

A solved grid g is an assignment of digits $\{1, \dots, 9\}$ to $N = 81$ cells. Its *coincidence matrix* $C \in \{0, 1\}^{N \times N}$ records which cell pairs share a digit: $C_{ij} = \mathbb{1}[g_i = g_j]$ for $i \neq j$. This matrix, flattened, is the “key source” for a stored memory.

A fixed random sparse matrix $W_{SH} \in \mathbb{R}^{H \times N^2}$ (density ρ , nonzero entries drawn i.i.d. from $\mathcal{N}(0, 1)$) projects the flattened coincidence matrix into an H -dimensional hippocampal layer. The engram is the top- K k -winner-take-all binarization of this projection:

$$h = \text{topK}(W_{SH} \text{vec}(C)) \in \{0, 1\}^H. \quad (1)$$

Each stored grid also has an associated *value* vector $b \in \mathbb{R}^N$, a per-cell digit-to-drive-level mapping.

Given M stored (key, value) pairs $\{(h_m, b_m)\}_{m=1}^M$ and a query engram h_q , the **attention readout** is

$$\hat{b} = \sum_{m=1}^M p_m b_m, \quad p = \text{softmax}\left(\frac{H_{\text{keys}} h_q}{\tau}\right), \quad (2)$$

a pure content-addressable lookup with no learned parameters beyond the fixed encoder. We compare this against a **ridge-regression readout**, which instead learns a single linear map W_{HD} (via the standard ridge solution) such that $W_{HD} h_m \approx b_m$ for all stored pairs, and retrieves via $\hat{b} = W_{HD} h_q$.

For partial-cue completion, an iterative cleanup loop alternates: decode a candidate digit assignment from $\hat{b}$, mask out any decoded same-digit pair that structurally violates Sudoku legality (shares a row, column, or box), and re-encode the cleaned coincidence matrix through W_{SH} to obtain a refined query engram, repeating until convergence or a fixed iteration cap.

2.2 Spiking network

The spiking network consists of $N = 81$ excitatory leaky-integrate-and-fire (LIF) neurons, one per cell, each paired with a fast inhibitory relay neuron. Membrane potential obeys

$$\tau_m \dot{v} = I_d + I_{\text{osc}}(t) - v, \quad I_{\text{osc}}(t) = 0.5 - 0.2 \cos(2\pi f_{\text{osc}} t), \quad (3)$$

with $\tau_m = 40$ ms, $f_{\text{osc}} = 10$ Hz (a shared oscillatory drive, common to every neuron), threshold $V_{\text{th}} = 1$, and reset to 0. I_d is a per-neuron DC current encoding a candidate digit identity: nine equally spaced levels in a range chosen so that every digit reliably crosses threshold once per oscillation cycle (Section 3.6).

The network’s *only* structural knowledge of Sudoku is the conflict mask Conf_{ij} , equal to 1 if cells i, j share a row, column, or box and 0 otherwise, and its complement. Conflicting cell pairs are coupled through the inhibitory relay (a fixed weight w_{IE}); non-conflicting pairs are coupled through direct weak excitatory synapses (weight w_{EE}). No synapse encodes which cells *should* share a digit – that information is never given structurally, only through the per-cell drive current, exactly mirroring the role of a query in the key-value architecture above.

3 Results

3.1 Combinatorial code: capacity and robustness

We swept the number of stored grids M (50–10,000), engram size K (10–500), and projection density ρ (0.002–1.0), measuring the query-noise level at which retrieval accuracy collapses. Table 1 reports

the collapse threshold (largest tested noise level, as a percentage of engram bits flipped, before top-1 accuracy falls off).

Table 1. Collapse-noise threshold, essentially flat across a 200-fold range of M and across K, ρ , except at very small K .

	$M=50$	$M=1000$	$M=10,000$	$K=10$	$K \geq 50$
Collapse threshold	45%	45%	40%	25%	40–45%

Robustness is essentially independent of M over a 200-fold range, and independent of connection density over $\rho \in [0.002, 1.0]$; the sole exception is engram sparsity itself – $K = 10$ is markedly less robust (25% vs. 40–45%) than any $K \geq 50$ tested, consistent with too few active units per engram leaving little redundancy to survive bit flips.

3.2 Key-value recall: attention vs. regression

Table 2 contrasts the two readouts at $M = 1000$ ($= H$) and $M = 2000$ ($= 2H$).

Table 2. Digit-cell retrieval accuracy, regression vs. attention readout.

	$M = 1000$		$M = 2000$	
Query noise	regression	attention	regression	attention
0%	99.98%	100.0%	21.8%	100.0%
2%	12.5%	100.0%	11.9%	100.0%
30%	11.3%	100.0%	11.1%	100.0%
50%	11.2%	11.7%	11.1%	10.7%

Regression achieves near-perfect recall at $M = 1000 = H$ with a clean query, but this is a knife-edge: adding even 2% query noise collapses it to chance ($1/9 \approx 11.1\%$), and at $M = 2000$ it is already at chance with *zero* noise – a hard capacity ceiling at $M \sim H$. Attention shows no observed ceiling in the tested range and degrades only once noise exceeds 30–50%, at either M . Figure 1 shows the full capacity curve.

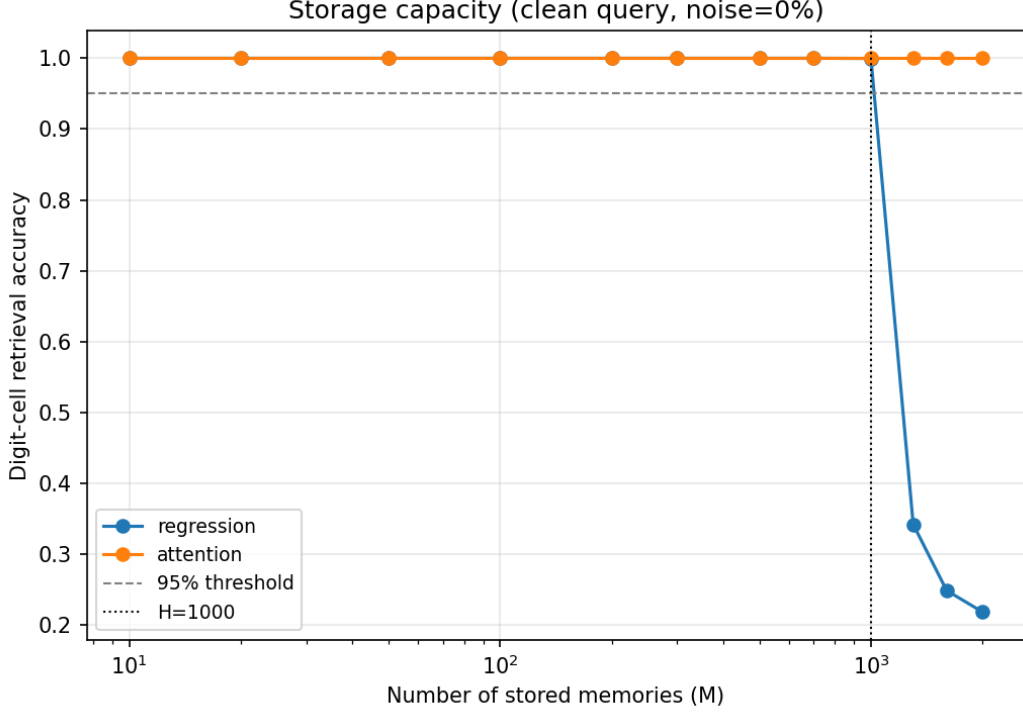


Figure 1. Estimated capacity (largest M retaining accuracy above a fixed threshold) vs. query noise, regression vs. attention readout.

3.3 Graceful degradation under query noise

Table 2 reports accuracy at a few noise levels; the full curve (Figure 2, left, $M = 200$) shows the shape of that degradation, and the benefit of the conflict-mask cleanup pass introduced in Section 2.1. Accuracy is perfect up to 35% of engram bits flipped, then declines smoothly – not catastrophically – over a narrow window before settling at chance by $\sim 48\%$. A finer sweep restricted to that window (Figure 2, right; 1 percentage-point steps, 100 trials/point) resolves the transition in detail and shows the cleanup pass’s rescue effect peaks sharply at 42% noise (59.3% vs. 42.5% without cleanup, a +16.8 percentage-point gain) and tapers off on both sides – negligible above 46%, where both curves have already collapsed to chance, and shrinking again below $\sim 39\%$, where the uncleaned query is already accurate enough that there is little left to rescue.

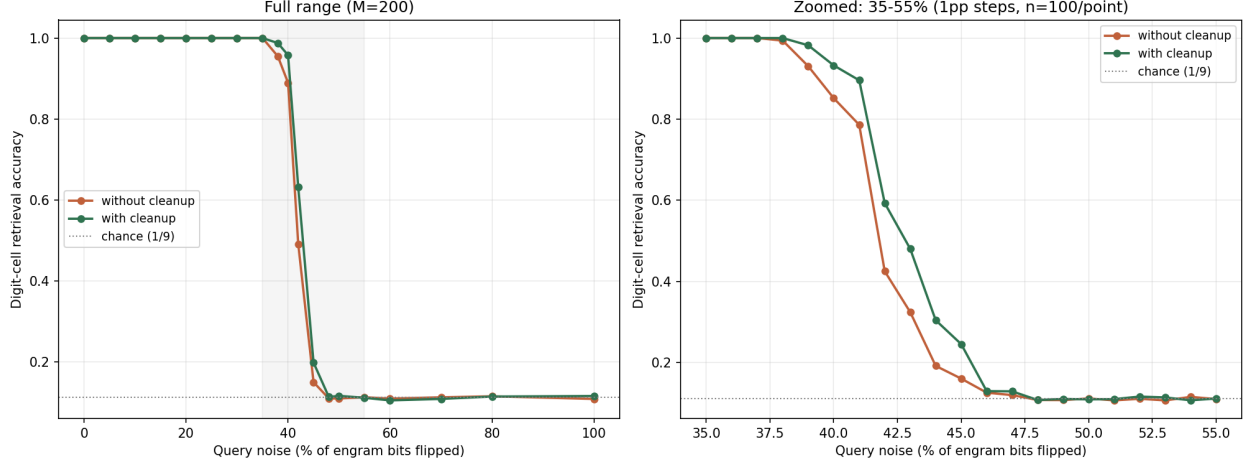


Figure 2. Digit-cell retrieval accuracy vs. query noise, with and without the conflict-mask cleanup pass. Left: full range (shaded band marks the zoomed window). Right: the 35–55% transition resolved at 1 percentage-point resolution, showing the rescue effect peaking at 42% noise.

3.4 Partial-grid pattern completion

Cueing the architecture with a partial grid (only some cells known) and running the conflict-mask attractor loop recovers missing digits with graceful, clue-count-dependent accuracy (Table 3, $M = 2000$, $K = 150$, $H = 1000$).

Table 3. Partial-grid completion: unknown-cell accuracy and exact whole-grid match, before (single-shot readout) and after (attractor loop) cleanup.

Clue fraction	Initial acc.	Final acc.	Initial exact	Final exact
0.20	13.0%	15.5%	1%	5%
0.25	14.7%	23.6%	1%	14%
0.30	25.1%	38.8%	6%	31%
0.40	59.1%	84.9%	38%	83%
0.50	93.7%	98.3%	88%	98%
0.70	100%	100%	100%	100%

Figure 3 shows the full curve; recovery is negligible below clue fraction ~ 0.15 , rises steeply through 0.20–0.40, and saturates by 0.5–0.7.

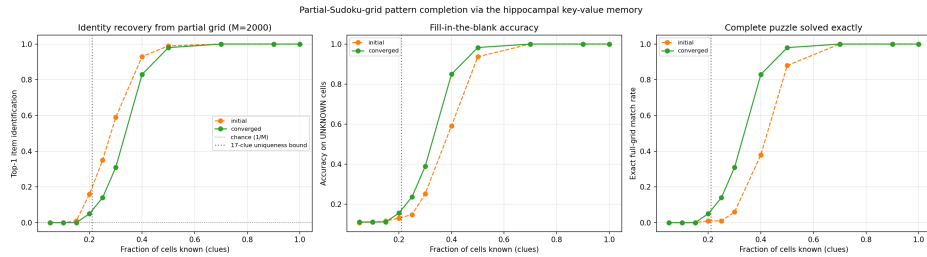


Figure 3. Partial-grid completion accuracy vs. clue fraction.

3.5 Relaxation dynamics near the completion threshold

The iteration count of the attractor loop, as a function of clue fraction, shows a reproducible dynamical signature reminiscent of relaxation-time crossovers studied in the statistical mechanics of random constraint satisfaction [7]. For the sharpest transition we found ($M = 500$, $K = 400$), a sub-critical plateau (92.6 ± 7.1 iterations, clue fraction ≤ 0.21) decays as a power law, $\tau \sim (p_c - p)^\nu$, to a floor of exactly 2 iterations by clue fraction 0.53. Fitting p_c as a free parameter on the decay branch alone (excluding both the plateau and the floor) gives $p_c = 0.395$ – matching the independently observed floor onset almost exactly – and $\nu = 1.32 \pm 0.08$ ($R^2 = 0.978$; Figure 4).

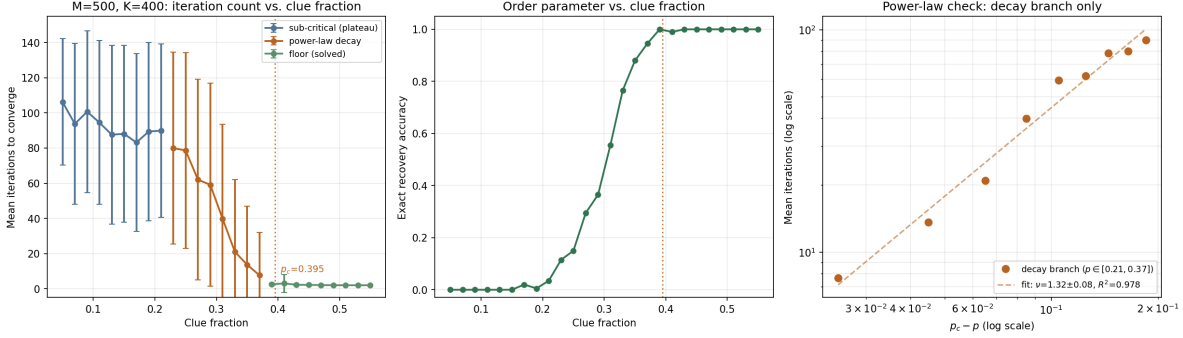


Figure 4. Relaxation-time crossover for the sharpest tested configuration ($M=500$, $K=400$): plateau, power-law decay, and floor regimes (left); order parameter (center); log-log power-law fit restricted to the decay branch alone (right).

We emphasize an honest limitation here. Repeating this fit for four additional (M, K) configurations gives exponents ranging from $\nu = 0.47$ to $\nu = 3.75$ – an eightfold spread too large to attribute to sampling noise alone. We do not have a validated universal exponent, and we report this rather than omit it: the crossover phenomenon itself replicates robustly (a clean power-law fit, $R^2 \geq 0.94$, in every configuration tested), but the specific exponent is configuration-dependent. We therefore treat Section 3.5 as a secondary, honestly-scoped characterization of the architecture’s dynamics, not as a claimed critical-phenomena universality result.

3.6 Spiking network: phase synchrony as an emergent conflict mask

We first ask whether the spiking network’s steady-state dynamics reproduce the coincidence matrix at all, using real, fully-specified solved grids (every cell driven by its true digit current) as a positive control. Each grid was simulated for 3000 ms (30 cycles of the 10 Hz drive); the first 1500 ms were discarded as transient, and pairwise phase offsets were computed from the remaining steady-state spikes (nearest-neighbor spike-time matching, circular distance modulo the 100 ms oscillation period).

At the operating point used throughout ($w_{IE} = 0.005$, $w_{EE} = 0.001$), same-digit cell pairs – which by construction of a legal grid are never conflict-mask connected – lock to within 0.205 ± 0.023 ms of each other, while differently-digitated pairs separate by ~ 13 ms, regardless of whether they happen to be inhibition-connected (12.97 ± 0.18 ms) or excitation-connected (13.00 ± 0.12 ms). This pattern is essentially identical across eight independently sampled grids (Table 4), and visibly forms a clean, reproducible “phase comb” each oscillation cycle (Figure 5).

Table 4. Steady-state phase offset by pair category, mean $\pm$ s.d. across 8 grids.

Pair category	Phase offset (ms)
Same digit (never conflict-connected)	0.205 ± 0.023
Conflicting, different digit (inhibition-connected)	12.97 ± 0.18
Non-conflicting, different digit (excitation-connected)	13.00 ± 0.12

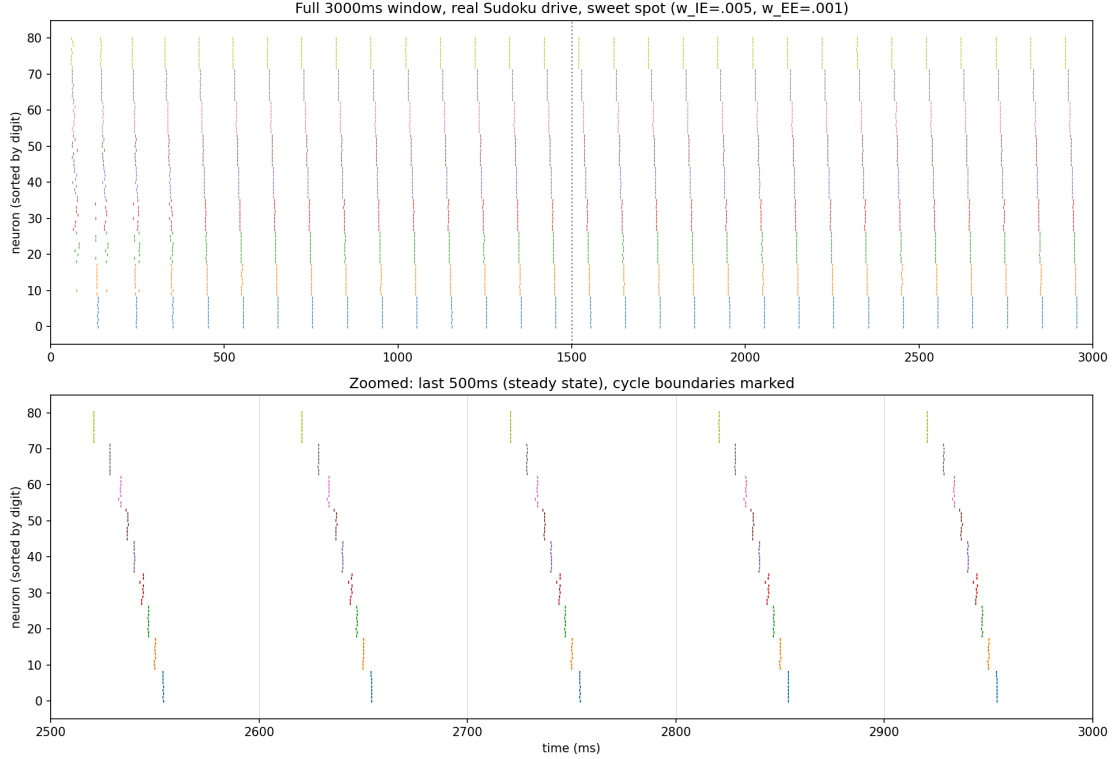


Figure 5. Spike raster, one representative grid, sorted by digit. Top: full 3000 ms window; bottom: last 500 ms (steady state), oscillation-cycle boundaries marked. Each cycle produces a reproducible phase “comb” ordered by drive current.

Because a legal grid never puts the same digit on two conflicting cells, the numbers above conflate the current-driven separation between different digits with any additional, connection-type-specific effect of inhibition. To isolate the latter, we assigned each cell a *random* digit label (ignoring legality) and measured the phase offset specifically between same-driven pairs, split by connection type. At the validated operating point, the inhibition-connected same-driven pairs show 2.0 ms more desynchronization than excitation-connected same-driven pairs, above a ~ 1 ms baseline gap present even with all coupling switched off (a structural artifact of the two pair categories, not a synaptic effect) – a real, if modest, causal contribution of the inhibitory wiring. This effect requires a specific weight balance: because each cell has roughly three times as many excitatory (complement-mask) partners as inhibitory (conflict-mask) partners, equal or near-equal per-synapse weights let the aggregate excitatory pull dominate and synchronize the *entire* population regardless of connection type, erasing the effect above $w_{EE} \gtrsim 0.003$.

We also explicitly tested, and ruled out, a stronger hypothesis: that conflicting pairs might settle into a genuine anti-phase lock (half-cycle separation), the mechanism underlying half-center oscillators in central pattern generators. Scanning inhibitory strength across an 80-fold range ($w_{IE} = 0.005$ to 0.4) produced no discernible trend toward anti-phase locking (steady-state offset flat at 3.4–3.6 ms throughout). This is a structural consequence of the shared oscillatory drive:

because every neuron receives an *identical* periodic forcing term, each oscillation cycle provides a fresh, drive-dominated threshold-crossing opportunity that a locally-scaled inhibitory kick cannot postpone past, regardless of its strength. Genuine anti-phase encoding would require either removing this common pacemaker or making inhibition strong enough to suppress firing for an entire cycle, neither of which holds in the present architecture.

Finally, we asked whether a fixed, small coincidence window could serve as a sharp same-digit/different-digit classifier on its own, independent of any downstream cleanup. Over a 0.5–10 ms threshold scan (four connectivity configurations, $n = 600$ pairs each), the best achievable balanced accuracy was 63–65%: real but soft, with substantial overlap between the two distributions (excitation-connected pairs’ 90th percentile offset, ~ 5 ms, exceeds inhibition-connected pairs’ 10th percentile, ~ 1.3 ms). A single pairwise check is not, by itself, a reliable decision rule.

3.7 Emergent coincidence matrix closes the loop

We now close the loop: rather than decoding a digit from any single neuron’s activity, we build an emergent coincidence matrix $\hat{C}$ directly from the network’s steady-state pairwise synchrony, thresholding each pair’s phase offset at $\theta = 2$ ms (the gap identified in Section 3.6). $\hat{C}$ is fed through the *same* W_{SH} projection, k -winner-take-all, and attention-readout pipeline used for storage and retrieval throughout this paper – the spiking network’s only role is to supply the relational input that the hard-coded conflict mask previously supplied.

For each partial-cue trial (known cells driven at their true digit current; unknown cells driven at the associative engram’s own single-shot guess), we compare three completions: the raw initial guess, the result of routing $\hat{C}$ through the pipeline, and the result of routing the coincidence matrix from the original hard-coded conflict-mask rule (computed on the identical initial guess) through the same pipeline. Table 5 reports coincidence-matrix quality and completion accuracy across 11 clue fractions ($M = 200$, $n = 25$ trials/point).

Table 5. Emergent coincidence matrix quality and downstream completion accuracy.

Clue fraction	$\hat{C}$ acc.	sens.	spec.	initial guess	dynamics pipeline
0.10	0.741	0.256	0.795	11.9%	12.1%
0.19	0.793	0.330	0.845	17.9%	23.4%
0.25	0.854	0.511	0.893	30.4%	54.8%
0.30	0.915	0.691	0.940	56.1%	75.6%
0.40	0.979	0.924	0.985	86.1%	90.4%
0.50	0.999	1.000	0.999	99.7%	100.0%

Specificity (correctly excluding truly-different-digit pairs) stays high throughout (≥ 0.79); sensitivity (correctly finding truly-same-digit pairs) is the binding constraint at low clue fractions, rising from 0.26 to 1.0 as clue fraction increases (Figure 6). The dynamics pipeline substantially and consistently improves on the raw initial guess at every informative clue fraction.

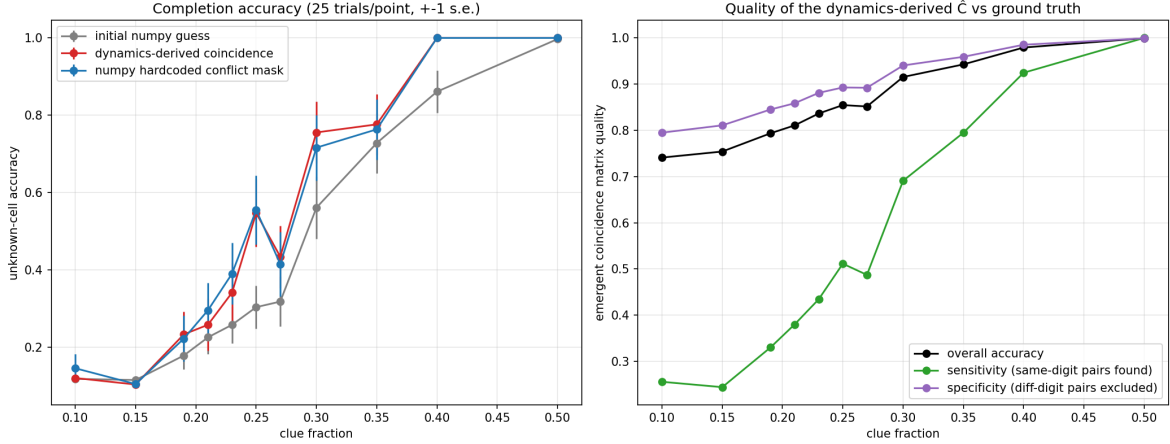


Figure 6. Left: completion accuracy vs. clue fraction, initial guess vs. dynamics-derived coincidence matrix vs. hard-coded conflict mask (25 trials/point, ± 1 s.e.). Right: quality of the emergent coincidence matrix vs. ground truth.

To test whether the dynamics pipeline matches or exceeds the hard-coded rule, we ran a second, statistically powered comparison in which all three methods were evaluated on *identical* trials (same target grid and known-cell mask), 20 trials per clue fraction across seven clue fractions, and applied a paired Wilcoxon signed-rank test. Table 6 reports the result: at every tested clue fraction, $p > 0.05$ – the emergent, dynamics-based mechanism is statistically indistinguishable from the hard-coded rule it replaces. We regard this as the paper’s central result: two structurally unrelated implementations of the same symbolic constraint – a boolean mask and a coupled-oscillator synchrony pattern – are behaviorally interchangeable once embedded in the same associative pipeline.

Table 6. Paired comparison, dynamics pipeline vs. hard-coded conflict mask, identical trials ($n = 20$ /point). No comparison reaches significance.

Clue fraction	0.15	0.19	0.21	0.25	0.27	0.30	0.35
Mean diff. (dyn. – rule)	+0.012	−0.048	−0.036	−0.043	+0.002	−0.001	+0.014
Wilcoxon p	0.23	0.55	0.89	0.33	0.48	0.87	0.29

We note, without over-interpreting a null result, that the same paired data show the dynamics pipeline’s improvement over the raw initial guess is concentrated in a minority of trials with large gains (a right-skewed distribution) rather than distributed uniformly; this is qualitatively consistent with genuine error-correction behavior – rescuing trials where the initial guess was substantially wrong – rather than a uniform accuracy shift, and is why the (large, obvious) mean improvement over the initial guess does not always translate into a formally significant paired test at $n = 20$.

3.8 Graceful degradation under drive-current noise

Two different kinds of robustness have been tested so far: corruption of the abstract engram vector itself (Section 3.3, random bit-flips) and missing per-cell information (Section 3.7, unknown cells driven by the associative engram’s own guess). Here we test a third, specific to the spiking mechanism: corruption of the physical drive current itself. Rather than any cell being entirely unknown, every *known* cell’s drive current was perturbed with Gaussian jitter before driving the network (clue fraction fixed at 0.30; unknown cells still use the associative engram’s own guess, unperturbed; $n = 20$ trials/point, paired across noise levels). Figure 7 shows the result: accuracy declines smoothly as current noise grows from 0 to over twice the spacing between adjacent digit

levels, with no sign of a cliff. Notably, even at the highest tested noise (σ equal to $2.2\times$ the inter-digit spacing – comparable to smearing a query halfway to its neighboring digit level on average) the dynamics pipeline (58.1%) still exceeds the noise-free symbolic-guess baseline it is compared against (51.9%): the mechanism does not merely tolerate drive noise gracefully, it remains a net improvement over the associative engram’s own single-shot guess even when its input is substantially corrupted.

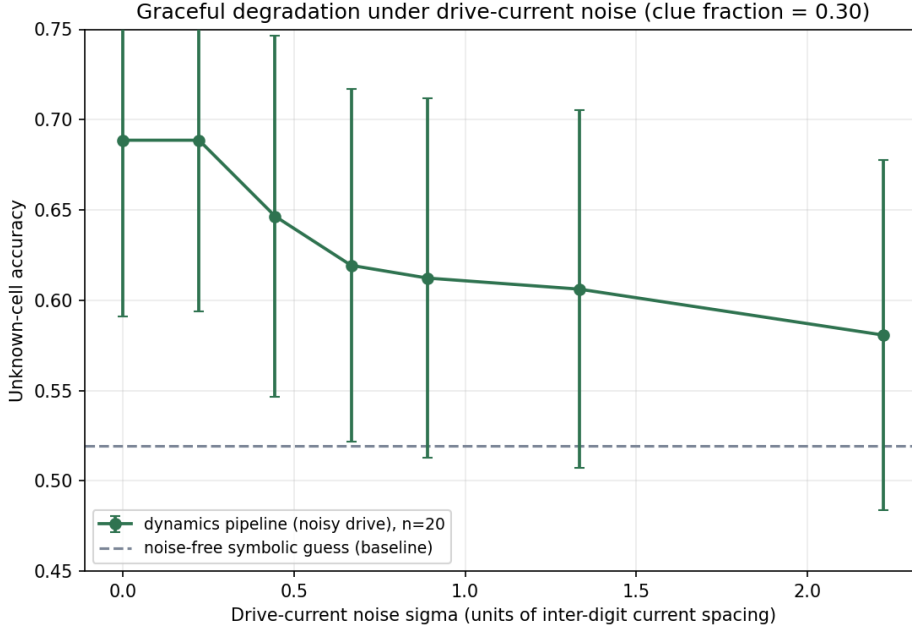


Figure 7. Unknown-cell accuracy vs. Gaussian drive-current noise (clue fraction 0.30), compared against the noise-free symbolic-guess baseline. Degradation is smooth, and the dynamics pipeline stays above baseline throughout the tested range.

3.9 Why low clue fractions are hard: signal quality, not signal quantity

A natural hypothesis for the low-clue-fraction bottleneck is that too few cells are confidently driven to produce enough coincident activity to strongly activate the correct stored engram. We tested this directly and found the opposite: at clue fraction 0.19, the emergent $\hat{C}$ has *more* active (same-digit-labeled) pairs on average (527) than the true grid (324) – the network is not under-active. The excess is almost entirely false positives (consistent with the specificity numbers in Table 5), and these spurious coincidences dilute the resulting engram’s ability to discriminate the correct stored memory among the M candidates: the softmax attention readout places only 37.6% of its mass on the true target at clue fraction 0.19 (with, on average, ~ 52 other stored grids scoring higher similarity), rising to 65.5% mass and a mean rank of 1.9 at clue fraction 0.30. The mechanism limiting low-clue-fraction performance is therefore noise in the query signal’s specificity, not a shortage of active units – a distinction that points at specificity (not raw signal strength) as the lever for improving this regime.

4 Extension: Cross-Modal Heteroassociation

Everything above stores and retrieves a single content type (Sudoku grids). Vector-HaSH [11] factors biological memory into two parts: a fixed, content-independent *scaffold* (grid-cell/hippocampal

recurrence) that provably generates exponentially many attractors with equally large basins, and a separate, plastic *heteroassociative* pathway linking arbitrary sensory input to that scaffold via “a Hebb-like online implementation of the pseudoinverse rule.” We test whether our architecture’s engram layer can play the scaffold’s role for a genuinely different modality – natural images – using exactly that heteroassociative mechanism, and we benchmark the result against classical Hopfield networks, whose well-known memory cliff Vector-HaSH is explicitly designed to avoid.

4.1 Architecture

An image $x \in \mathbb{R}^D$ is linked to its paired grid’s engram $h \in \{0, 1\}^H$ by two ridge-regularized pseudo-inverse matrices, W_{SI} (image→engram) and W_{IS} (engram→image), trained as a *bidirectional, one-memory-per-pair* association – not many exemplars regressed onto one shared class target, which we found generalizes poorly (direct engram-overlap measurement showed it collapsing from 0.835 to 0.330 between $\sigma = 0$ and $\sigma = 0.10$ pixel noise, versus a smooth $1.0 \rightarrow 0.869$ decay for the bidirectional rule at matched noise). Recall is a four-stage loop: a noisy image cue is projected through W_{SI} and thresholded to an initial engram; the existing attention readout and conflict-mask cleanup (Section 2.1) iterate this engram to a Sudoku-consistent fixed point, exactly as for a partial-grid cue; the cleaned engram is projected back through W_{IS} to reconstruct the image. Figure 8 shows representative cases where this cleanup step turns a badly corrupted initial reconstruction into an exact one.

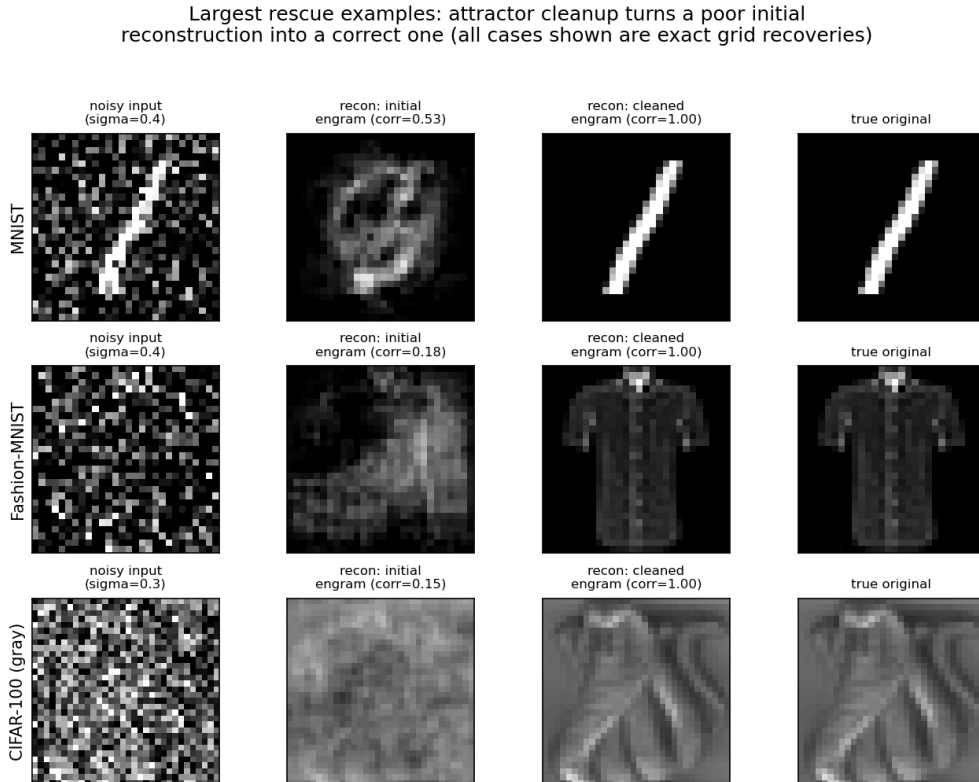


Figure 8. Representative cleanup examples (one dataset per row-group): noisy input, image reconstructed from the engram *before* cleanup, image reconstructed *after* the Sudoku-attractor pass, and the true original. All cases shown are exact grid recoveries.

4.2 Generalization across datasets

Table 7 sweeps corruption at $M=200$, $H=1000$, $K=150$ across three datasets of increasing dissimilarity from the original: MNIST (28×28), Fashion-MNIST (same dimension, unrelated content), and CIFAR-100 grayscale (32×32 , different dimension *and* content). The qualitative shape – a plateau, then a smooth decline, no cliff – replicates across all three; CIFAR-100 declines somewhat earlier, consistent with its images carrying less redundant, more textured content that a fixed-density sparse code compresses less cleanly.

Table 7. Exact grid recovery vs. image noise, three datasets ($M=200$, $n=200/\text{point}$, Wilson 95% CI).

σ	MNIST	Fashion-MNIST	CIFAR-100 (gray)
0.30	99.5% [97.2,99.9]	97.0% [93.6,98.6]	90.0% [85.1,93.4]
0.40	82.0% [76.1,86.7]	85.0% [79.4,89.3]	67.0% [60.2,73.1]
0.50	49.5% [42.6,56.4]	58.5% [51.6,65.1]	36.0% [29.7,42.9]
0.60	29.5% [23.6,36.2]	31.5% [25.5,38.2]	17.5% [12.9,23.4]
1.00	4.0% [2.0,7.7]	5.0% [2.7,9.0]	3.5% [1.7,7.0]

4.3 A trivial baseline this architecture does not yet beat

At $M=200$, a reviewer’s natural objection is that this may reduce to nearest-neighbor classification into 200 buckets. We tested this directly: for the identical noisy queries used above, a raw-pixel Euclidean nearest-neighbor classifier among the M stored clean images predicts which grid is present, and we compared its exact-match rate against ours with a paired McNemar test. The answer is not reassuring: nearest-neighbor performs *at least as well, and significantly better*, across nearly the entire informative range. At $\sigma=0.40$, MNIST nearest-neighbor is still at 100% while our pipeline is at 82.0% (McNemar $p < 0.0001$); at $\sigma=1.00$ nearest-neighbor remains at 95.0% while ours has fallen to 4.0%. The pattern replicates on Fashion-MNIST and CIFAR-100 (Table 7 vs. the corresponding nearest-neighbor rates, all $p < 0.0001$ except two borderline points at the highest tested noise). At $M=200$ with well-separated exemplars, Euclidean nearest-neighbor is close to the likelihood-optimal classifier under additive Gaussian corruption; routing through a lossy, sparsely-thresholded engram throws away information the raw pixels still carried. We report this plainly rather than omit it: whatever this architecture’s value, it is not yet raw classification accuracy at small M . Its case has to rest on properties nearest-neighbor search structurally lacks – a fixed-size, compressed, content-addressable code that never revisits the stored exemplars, and (Section 4.5) a qualitatively different failure mode under scale than classical attractor memories.

A natural hope is that this gap closes as M grows, since nearest-neighbor search among more candidates should itself get harder. We tested this directly at fixed $\sigma=0.4$, scaling M from 200 to 2000 with the same properly-regularized weights as Section 4.4 ($\lambda=1.0$). It does not close – it widens sharply:

Table 8. Does the nearest-neighbor gap close as M grows? ($\sigma=0.4$, $n=200/\text{point}$.)

M	Ours	k -NN	Gap (k -NN – ours)
200	96.5%	100.0%	+3.5pp
500	48.5%	100.0%	+51.5pp
800	20.5%	100.0%	+79.5pp
1200	9.0%	100.0%	+91.0pp
2000	3.5%	99.5%	+96.0pp

Nearest-neighbor’s accuracy is nearly flat across the entire range ($100\% \rightarrow 99.5\%$), while ours continues to decline under the same two capacity ceilings characterized next (Section 4.4). Every gap is significant at $p < 0.0001$ (paired McNemar) except $M=200$ ($p=0.016$). This settles the question in the least favorable direction: whatever this architecture is for, it is not for beating a nearest-neighbor lookup at any M we tested.

4.4 Capacity: two independent ceilings, one of them fixable

Scaling M from 200 to 2000 (fixed $H=1000$, default ridge $\lambda=10^{-4}$) reproduces what looks, at first, like a hard cliff: exact recovery at $\sigma=0.1$ falls from 100% ($M \leq 300$) to 8.3% at $M=500$. Measuring the image-side Gram matrix directly ($XX^\top$, X the $M \times D$ image matrix) explains why: its condition number grows from 1.8×10^4 ($M=200$) to 3.6×10^7 ($M=500$), crossing the fixed ridge value exactly where the collapse occurs. Re-solving with $\lambda=0.1$ at $M=500$ restores 100% exact recovery at $\sigma=0.1$ while costing almost nothing in zero-noise fidelity (0.994 vs. 1.000 overlap) – this specific cliff is a stale hyperparameter, not an architectural limit.

A second, genuinely structural ceiling remains underneath it. The image Gram matrix’s *effective rank* – not its nominal pixel dimension $D=784$ – caps exact zero-noise fitting: for this MNIST sample it saturates at ~ 600 – 610 regardless of how large M grows (rank $568/800$ at $M=800$, only reaching $607/2000$ by $M=2000$), a property of the data’s intrinsic dimensionality that no amount of H or ridge tuning can lift. CIFAR-100 grayscale, by contrast, fills its *entire* ambient dimension (rank $1024/1024$ by $M=1600$) – natural texture carries more independent information per pixel than a clean digit stroke on a black background, though this does not translate into better absolute recovery (Table 7). A separate, H -dependent ceiling governs the *reverse* (engram $\rightarrow$ image) direction: its Gram matrix’s rank is bounded by $\min(M, H)$, and doubling H from 1000 to 2000 (density held fixed) measurably delays this one – exact recovery at $M=2000$, $\sigma=0.1$ rises from 80% ($H=1000$) to 90% ($H=2000$) – with zero effect on the image-side ceiling, exactly as the algebra predicts.

Does the ridge value that fixes $M=500$ keep working, or does λ need to keep growing with M ? We extended the sweep to $M=2500$ – 4000 across $\lambda \in \{1, 10, 50, 100\}$:

Table 9. Exact recovery at $\sigma=0.1$ vs. ridge λ , extended M range ($n=50$ /point).

M	$\lambda=1$	$\lambda=10$	$\lambda=50$	$\lambda=100$
2000	72%	74%	–	–
2500	56%	62%	52%	32%
3000	50%	64%	40%	18%
4000	28%	38%	22%	16%

$\lambda \approx 10$ stays at or near the best choice across the entire $M=500$ – 4000 range tested – pushing it higher ($50, 100$) actively hurts rather than helps, so the required ridge does *not* keep growing without bound; a single, fixed value is genuinely reusable. The residual decline past $M=2000$ even at this best-tuned λ ($74\% \rightarrow 64\% \rightarrow 38\%$) is not a ridge problem at all: by $M=3000$ – 4000 one is 5 – $7\times$ past the image effective-rank ceiling already identified above, and no amount of regularization can recover information the images never carried in the first place.

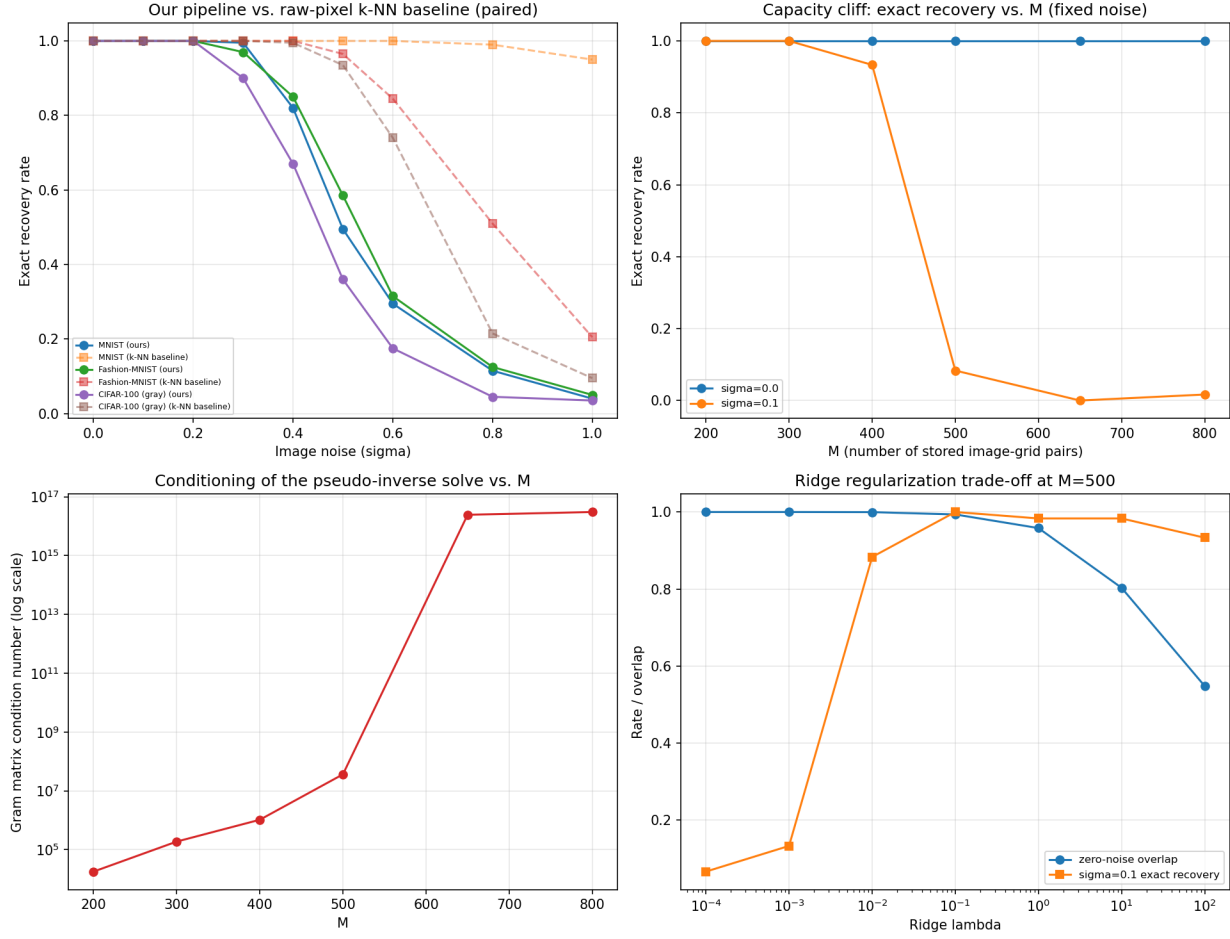


Figure 9. Top-left: our pipeline vs. the nearest-neighbor baseline (paired). Top-right: the $M=500$ cliff under the default ridge. Bottom-left: image Gram-matrix condition number vs. M (log scale), crossing the ridge value exactly where the cliff appears. Bottom-right: ridge sensitivity at $M=500$, showing the fix.

4.5 Comparison to classical Hopfield networks

Vector-HaSH’s stated motivation is avoiding “the memory cliff of prior memory models” [11]. We tested this claim directly rather than citing it, building four auto-associative memories on the *same* bipolarized MNIST images and comparing them against our full heteroassociative-plus-cleanup pipeline: (i) the classical outer-product (Hebbian) rule; (ii) a mean-centered covariance rule correcting for population-level pixel bias; (iii) the pseudo-inverse (projection) rule – the same learning-rule family our own network uses, but with no separate error-correcting scaffold; and (iv) our network. As a sanity check on the classical rule’s textbook behavior, we first reproduced its capacity cliff on *random* bipolar patterns ($N=784$): exact recovery falls from 82.5% ($M=60$) to 0% ($M=120$), matching the analytic prediction ($M/N \approx 0.138$) almost exactly.

On real, correlated images the picture is markedly worse for every auto-associative variant. The classical rule fails even at *zero* noise, for any M we tested: a clean cue’s overlap with itself drops from 1.0 to 0.84 after a single update, because 86% of MNIST pixels are background and the plain Hebbian rule’s dominant mode locks onto that population bias rather than any specific image. Mean-centering removes this bias exactly as intended – we verified it recovers correctly at $M \leq 3$ –

but real images correlate with each other far beyond a first-moment bias, and this corrected rule still collapses to 0% by $M=10$, two orders of magnitude below the random-pattern prediction. The pseudo-inverse rule is the most instructive comparison, because it is the same rule ours uses: under raw random bit-flips it is algebraically *exact* at zero noise up to at least $M=200$, but its basins of attraction shrink so fast under even modest noise that 5% bit-flip recall falls from 100% ($M=100$) to 25% ($M=150$) to 0% ($M=200$).

The comparison above used two different corruption models – bit-flips for the three auto-associative baselines, Gaussian pixel noise for ours – with no shared severity axis. We redid it with a single, shared model: every method sees the same Gaussian-corrupted continuous image per trial, and the three Hopfield variants receive it bipolarized (thresholded at 0.5), so their bit-flips arise naturally from the same noise process rather than being imposed separately. At $n=200/\text{point}$ (matching Table 7’s rigor):

Table 10. Exact recovery vs. M on identical MNIST images, single shared Gaussian noise model ($\sigma=0.3$, $\sim 7\%$ mean effective bit-flip; $n=200/\text{point}$).

M	Classical	Covariance	Pseudo-inverse	Our network
100	0%	0%	91.5%	100.0%
200	0%	0%	0%	99.5%
400	0%	0%	0%	94.0%
800	0%	0%	0%	57.5%

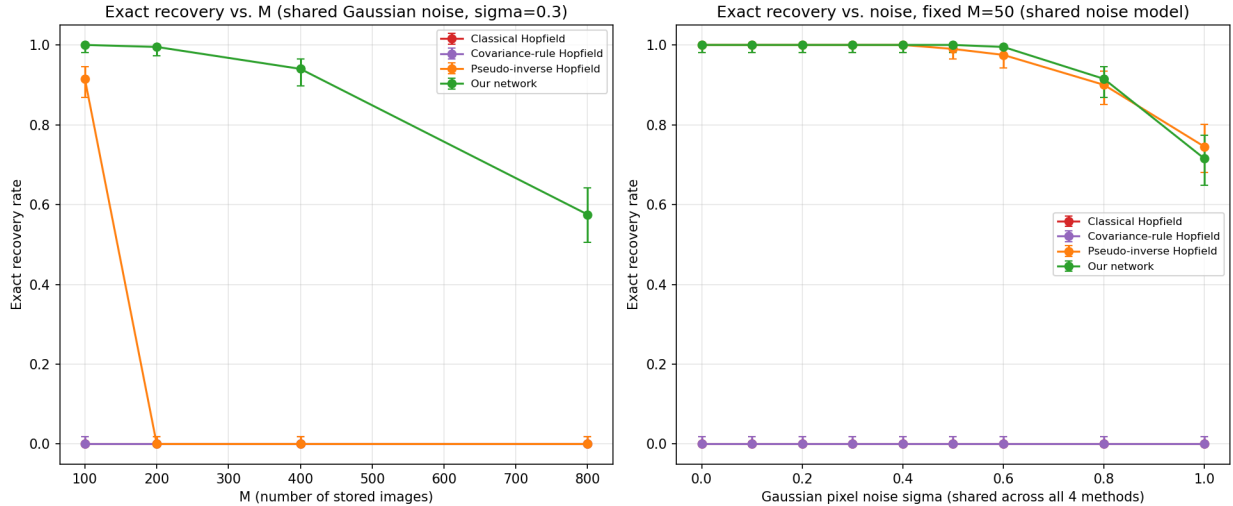


Figure 10. Left: exact recovery vs. M , all four methods, shared Gaussian noise ($\sigma=0.3$), Wilson 95% CIs. Right: exact recovery vs. noise at fixed $M=50$, same shared noise model throughout.

The capacity-scaling story is unchanged, and now more sharply confirmed: pseudo-inverse Hopfield collapses to exactly 0% by $M=200$, while ours holds 57–99% out to $M=800$. But testing the shared model across noise levels at a fixed, safe $M=50$ reveals something the earlier mismatched comparison could not show cleanly: at fixed M , our network and the pseudo-inverse baseline are statistically comparable across the entire noise range, and the pseudo-inverse rule is marginally *ahead* at the highest noise tested (74.5% vs. 71.5% at $\sigma=1.0$). Our network’s real advantage over the identical learning rule is not noise tolerance at fixed M – on that axis they are essentially tied – it is capacity scaling: the two share equal footing until M crosses somewhere between 100 and 200, past which only the version with a separate error-correcting attractor survives. This is a

narrower and more precise claim than a blanket “more robust” would have been, and it is the one the shared-noise-model data actually supports.

Because the pseudo-inverse baseline uses the identical learning rule to ours, this isolates the source of the capacity difference precisely: not the pseudo-inverse rule itself, but the presence of a separate, content-specific error-correcting step (the Sudoku-legality attractor) layered on top of it. This is the piece Vector-HaSH’s true scaffold provides in a content-*independent* way (Section 2); ours is content-dependent (legality-specific) but still delivers the qualitative signature the scaffold is meant to produce, and does so an order of magnitude beyond where the shared learning rule alone can reach.

5 Discussion

Three results compose into a single picture. The combinatorial engram code is robust across a 200-fold range of storage load and independent of connection density (Table 1); the attention-based key-value readout built on top of it shows no observed capacity ceiling where a regression-based readout collapses catastrophically (Table 2); and the symbolic legality constraint this architecture relies on for cleanup can be realized as an emergent property of a spiking network’s phase-synchrony dynamics, statistically matching – not exceeding – the hard-coded rule (Table 6). We regard the “matches, not exceeds” framing as the honest and more interesting finding: it demonstrates that a biologically-plausible, locally-wired dynamical mechanism can substitute for a piece of hard-coded logic without loss, which is a different and, we think, more useful claim than a performance improvement would have been.

A theme recurs across every result in this paper: failure, when it happens, is graceful rather than catastrophic. The combinatorial code and its attention readout hold up to 30–35% query corruption before collapsing over a narrow window rather than failing sharply (Section 3.3); partial-grid completion degrades smoothly with clue count instead of failing at a hard threshold (Table 3); and the spiking mechanism tolerates substantial drive-current noise, remaining a net improvement over the symbolic guess even when corrupted well past the spacing between adjacent digit levels (Section 3.8). Where a cleanup pass exists, its benefit is not uniform but concentrates specifically in the transition region – the conflict-mask cleanup’s rescue effect on query noise peaks sharply at 42% (Figure 2, right) – exactly where ambiguity is highest and correction is most needed, and negligible everywhere else, either because the query was already easy or already unrecoverable. The cross-modal extension (Section 4) reinforces this theme with an external, hard baseline: on identical real images, three classical/pseudo-inverse auto-associative memories all collapse to exactly 0% recovery within a narrow range of stored-item count, while our architecture – sharing the pseudo-inverse learning rule with one of them, but adding a separate legality-based cleanup step – degrades smoothly and remains above 0% an order of magnitude further out (Table 10). This is the same qualitative signature Vector-HaSH’s content-independent scaffold is built to produce, obtained here from a content-*dependent* one.

We invoke the statistical mechanics of constraint satisfaction and random k -SAT [7, 8] as motivating context for why constraint density and dynamics near a solvability boundary are worth studying together – and, relatedly, why McGuire et al.’s proof that no 16-clue Sudoku has a unique solution [9] is a natural reference point for combinatorial threshold phenomena in this domain – but we do not claim to extend that literature’s universality results. Section 3.5’s honestly reported non-universal exponent is the reason for this restraint, and we consider explicitly reporting a negative/mixed finding here preferable to omitting it.

Limitations. The architecture’s shared, common oscillatory drive is structurally incompatible

with anti-phase-based constraint encoding (Section 3.6); this is a property of this specific circuit, not a general claim about spiking constraint satisfaction. The low-clue-fraction regime remains limited by coincidence-matrix specificity, and we have not tested whether a smarter (e.g., adaptive) synaptic mechanism could improve it without compromising the clean matching result of Table 6 – we deliberately did not pursue synaptic plasticity in the reported architecture, since the non-plastic, fixed-weight dynamics already achieve parity with the symbolic rule, and we judged added mechanism without added validated benefit to be the wrong trade for this paper.

The cross-modal extension has its own, separately reported limitations, three of which we tested directly rather than leaving open. First, it is not yet more accurate than a trivial raw-pixel nearest-neighbor classifier at $M=200$ (Section 4.3), and we checked whether this improves at the large- M scale where a fixed-size engram’s storage advantage should start to matter – it does not: the gap widens from +3.5 percentage points at $M=200$ to +96.0 points at $M=2000$ (Table 8), since nearest-neighbor accuracy is nearly insensitive to M while ours is not. Whatever this architecture is for, it is not for beating nearest-neighbor lookup at any M we tested; its case would have to rest on properties outside classification accuracy entirely (e.g. fusing partial pixel and partial relational cues, which nearest-neighbor over images alone cannot do). Second, the ridge-regularization question is resolved rather than open: extending the sweep to $M=4000$ (Table 9) shows the required λ does *not* keep growing – a single $\lambda \approx 10$ stays near-optimal throughout – and the residual decline past $M=2000$ is fully attributable to the already-identified image effective-rank ceiling, not to unresolved regularization. Third, the Hopfield comparison’s mismatched noise models (bit-flip vs. Gaussian) have been replaced with a single shared model (Section 4.5), which revealed a more precise finding than the original qualitative one: at fixed, small M our network and the pseudo-inverse baseline are statistically comparable in noise tolerance – the real advantage is capacity scaling, not noise robustness per se.

6 Methods

All simulations use Brian2 [10]. Spiking network parameters: $\tau_m = 40$ ms, $f_{\text{osc}} = 10$ Hz, $V_{\text{th}} = 1$, $V_{\text{reset}} = 0$, integration timestep 0.1 ms (RK4), $w_{IE} = 0.005$, $w_{EE} = 0.001$ (Section 3.6 operating point). Digit-encoding drive currents use nine evenly spaced levels chosen so every level reliably crosses threshold at least once per oscillation cycle (below ~ 0.46 , a level never fires; above this floor, separation in first-spike phase is monotonic in current). Simulation window 3000 ms per trial, with the first 1500 ms discarded as transient; every trial builds a fresh network instance (Brian2’s absolute simulation clock does not reset on a reused network object, so a persistent network across trials would confound successive trials’ oscillation phase). Initial membrane potential is randomized per trial (Uniform(0, 0.1) + Uniform(−0.1, 0.1) perturbation).

Engram/readout parameters: $H = 1000$, $K = 150$ (capacity/robustness sweeps) or $K = 400$ (crossover characterization, chosen as the sharpest transition in an earlier M – K grid search), projection density $\rho = 0.02$, softmax temperature $\tau = 2.0 \times (K/150)$ (scaled to the engram’s self-overlap). Statistical tests: two-sided Wilcoxon signed-rank (`scipy.stats.wilcoxon`) on paired per-trial accuracy differences, identical target grid and known-cell mask across compared methods within each trial.

Heteroassociation (Section 4): W_{SI}/W_{IS} trained via the dual-form ridge pseudo-inverse ($\lambda = 10^{-4}$ default, $\lambda = 0.1$ – 10 where noted for larger M , verified stable up to $M=4000$), images normalized to $[0, 1]$ and Gaussian noise added and reclipped to that range. MNIST/Fashion-MNIST are 28×28 (dimension 784); CIFAR-100 is converted to grayscale by averaging RGB channels (32×32 , dimension 1024). Baseline (Section 4.3): paired McNemar exact test (`scipy.stats.binomtest`

on discordant-pair counts) between our pipeline’s and nearest-neighbor’s exact-match outcome on identical noisy queries, at $M=200$ and, separately, across $M=200\text{--}2000$ at fixed $\sigma=0.4$. Hopfield baselines (Section 4.5): images bipolarized by thresholding at 0.5; classical rule is the outer-product/Hebbian sum with zero diagonal; covariance rule subtracts each pixel’s own mean activity (not a single global scalar) before the same sum; pseudo-inverse rule reuses the identical dual-form solver as W_{SI}/W_{IS} with the pattern set as both input and target. Synchronous recall update $s \leftarrow \text{sign}(Ws)$ preserves a neuron’s current state on an exact tie (net input 0) rather than forcing it to +1, since mean-centered patterns can have exactly-zero weights on pixels that are constant across every stored image. The headline Hopfield comparison (Table 10) uses a single shared noise model across all four methods: a Gaussian-corrupted continuous image is generated once per trial, consumed directly by our network and bipolarized (thresholded at 0.5) for the three Hopfield variants, at $n=200/\text{point}$ with Wilson 95% CIs; an earlier bit-flip-vs-Gaussian mismatched version is reported separately in the text as a mechanism illustration only.

Appendix: M , K , H by Figure

H (hippocampal/engram dimension) and K (active units per engram) are 1000 and 150 throughout this paper unless noted otherwise – never tuned per figure. M (number of stored items) is the one parameter that varies, either held fixed or swept as the figure’s own x -axis. Table 11 gives the exact values for every figure, read directly from each figure’s generating script rather than reconstructed from memory.

Table 11. M , K , H for every figure in this paper.

Fig.	Generating script	M	K	H
1	capacity_regression_vs_attention.py	10–2000 (swept, x -axis)	150	1000
2	hippocampus_feedback_noise_zoomed.py	200	150	1000
3	sudoku_partial_grid_completion.py	2000	150	1000
4	critical_slowing_down_clean_plot.py	500	400	1000
5	spiking_real_sudoku_sweetspot.py	–	–	–
6	spiking_emergent_coincidence.py	200	150	1000
7	spiking_drive_noise_robustness.py	200	150	1000
8	heteroassoc_vectorhash_best_examples.py	200	150	1000
9	heteroassoc_final_consolidation.py	see note	150	1000
10	hopfield_matched_noise_rigorous.py	see note	150	1000

Figure 5 is a single-grid spiking simulation ($N=81$ cells) with no engram storage or retrieval involved, so $M/K/H$ do not apply. Figure 4 uses $K=400$ – the sharpest crossover found in an earlier M – K grid search – rather than the default 150. Figures 9 and 10 each have panels at different M : Figure 9’s top-left (pipeline vs. nearest-neighbor) and top-right/bottom-left (capacity cliff and Gram-matrix conditioning) panels use $M=200$ and $M \in \{200, 300, 400, 500, 650, 800\}$ respectively, and its bottom-right (ridge sensitivity) panel fixes $M=500$; Figure 10’s left panel (M -sweep) uses $M \in \{100, 200, 400, 800\}$ at fixed $\sigma=0.3$, and its right panel (noise-sweep) fixes $M=50$.

Code and Data Availability

All simulation code, raw data (CSV), and figure-generation scripts are available at <https://github.com/ghub-res1578/sudoku-hippocampal-key-value-memory>, including a README with exact commands to regenerate every table and figure in this paper.

References

- [1] D. Marr, “Simple memory: a theory for archicortex,” *Phil. Trans. R. Soc. Lond. B* **262**, 23–81 (1971).
- [2] E. T. Rolls, “The mechanisms for pattern completion and pattern separation in the hippocampus,” *Front. Syst. Neurosci.* **7**, 74 (2013).
- [3] J. J. Hopfield, “Neural networks and physical systems with emergent collective computational abilities,” *PNAS* **79**, 2554–2558 (1982).
- [4] D. J. Willshaw, O. P. Buneman, H. C. Longuet-Higgins, “Non-holographic associative memory,” *Nature* **222**, 960–962 (1969).
- [5] D. Krotov, J. J. Hopfield, “Dense associative memory for pattern recognition,” *NeurIPS* (2016).
- [6] H. Ramsauer et al., “Hopfield networks is all you need,” *arXiv:2008.02217* (2020).
- [7] M. Mézard, G. Parisi, R. Zecchina, “Analytic and algorithmic solution of random satisfiability problems,” *Science* **297**, 812–815 (2002).
- [8] M. Mézard, A. Montanari, *Information, Physics, and Computation*, Oxford University Press (2009).
- [9] G. McGuire, B. Tugemann, G. Civario, “There is no 16-clue Sudoku: solving the Sudoku minimum number of clues problem via hitting set enumeration,” *Experimental Mathematics* **23**, 190–217 (2014).
- [10] M. Stimberg, R. Brette, D. F. M. Goodman, “Brian 2, an intuitive and efficient neural simulator,” *eLife* **8**, e47314 (2019).
- [11] S. Chandra, S. Sharma, R. Chaudhuri, I. Fiete, “Episodic and associative memory from spatial scaffolds in the hippocampus,” *Nature* **638**, 739–752 (2025).